# PA3 – Tabular Q-Learning (Nim)

Due: See Gradescope

## Overview

In this assignment you will implement tabular Q-learning for Nim, a two-player turn-based strategy game. You will train an RL policy through simulated play against simple opponents, then evaluate the learned greedy policy. All graded code tasks are marked with TODO comments in `PA3.ipynb`. Question 4 is graded from the written response prompts in the notebook.

## Learning Goals

- Represent game states and legal actions for Nim.
- Implement tabular Q-learning updates and epsilon-greedy action selection.
- Build an episode-level training loop against random and rule-based opponents.
- Evaluate a learned greedy policy over many matches.
- Interpret a Q-table, a greedy action, and a reported win rate.

## Rules and Allowed Libraries

- Use only the packages already imported in the notebook.
- Do not use external RL libraries, such as Gymnasium or Stable-Baselines3.
- All RL logic must be implemented from scratch using Python standard libraries.
- GenAI tools that generate output may be used in course assignments (not tests or exams) to support your learning, as long as you do so honestly through proper documentation, citation, and acknowledgement.

## Deliverables

- Submit `PA3.ipynb` with all TODOs in Questions 0–3 completed and the Question 4 written responses filled in.
- Submit the notebook to Gradescope as an `.ipynb` file, not as a PDF.
- Do not rename required functions or change function signatures.

## Point Distribution (100 total)

| Section | Points |
|---|---|
| Question 0: Nim setup and utilities | 15 |
| Question 1: Core Q-learning functions | 40 |
| Question 2: Training loop | 25 |
| Question 3: Policy and evaluation | 15 |
| Question 4: Short written responses (manually graded) | 5 |

## Optional Extension

Question 5, the SARSA extension, is optional and ungraded. It is included for extra practice comparing an on-policy update with the Q-learning update.

## Notes

- The autograded portion of PA3 is worth 95 points total.
- Question 4 is graded manually from the written responses in the notebook.
- Gradescope will show some autograder checks immediately after submission, and additional autograder checks also run during grading but are not shown in advance.
- Reward is always from player 0's perspective: +1 win, -1 loss, 0 otherwise.
- Player 0 always moves first.
- Suggested debug baseline: `episodes=10000`, `alpha=0.5`, `gamma=0.9`, `epsilon=0.1`.

## Submission Checklist

- All required TODOs in Questions 0–3 are implemented.
- Question 4 written responses are filled in.
- Notebook runs end-to-end without errors.
- You did not rename required functions or change function signatures.
- Your `.ipynb` file is uploaded to Gradescope and the autograder shows no submission errors.

## Tips

- Complete Question 0 first; most later functions depend on these utilities.
- In Question 2, update the agent's last `(state, action)` after the opponent moves, not immediately after the agent moves.
- Run each nearby checkpoint cell after finishing a TODO.
- Submit after each question, because earlier functions are used in later ones.